

9.5 第0讲 课程简介

rentw@nju.edu.cn

邮件 COA 以...

深入理解计算机系统 3版 英文版 网上公开

分数 平时 { 书面 10%
编程 20%

按时且全答 不会写不会

see coder 分数

期中上机 30%

2次取最高 来自于平时作业 认真!

期末书面 40%

网评卷 10%~20% 较难 看平时踏实 题量大

(奖励 10%)

这门课难在哪里? 计算机各元素均衡发展(木桶理论)

补偿游戏 (理解课程的关键)

第1讲 计算机系统概述

一 组织 ^{Organization} 对程序员不可见
结构 ^{Architecture} 可见

操作单元及其相互连接

直接影响程序执行属性

指令集 表示数据类型的位数

二 计算机发展史 (不考)

1. 冯诺依曼结构 “存储程序”思想 输入/输出 存储器 处理单元 控制单元

Q1. 为什么存储程序?

① 操作 → 二进制 ② 存入计算机

ENIAC 手动编程 慢

Q2. 如何区分数据和指令?

根据地址区分

2. 真空管 → 晶体管 → 集成电路

“摩尔定律”

→ 快

小

低造价

普及

↓

↑ 催生
移动设备

移动互联网 (定位能力)

↓ 减少对电能消耗和散热的需求

内蒙(风电)

冷却? ① 冰岛 ② 水(贵州)

三 计算机性能

计算机设计的主要目标是提高 CPU 性能

不能无限提高 “多核”

系统时钟 时钟频率 Hz 3.8 G (Hz)

13 最基础操作的次数

时钟周期 s

时钟滴答

指令执行 CPI 每条指令需要多少时钟周期

$$T = I_c \times CPI \times t$$

MIPS 每秒执行几百条指令

MFLOPS ... 浮点操作

基准程序

- 基准程序
 - 使用一系列基准程序来测量系统的性能
 - 平均结果:
 - 算数平均值: $R_A = \frac{1}{m} \sum_{i=1}^m R_i$
 - 调和平均值: $R_H = \frac{m}{\sum_{i=1}^m \frac{1}{R_i}}$

第二讲 计算机的顶层设计

一 计算机的不同视图

外围设备

顶层结构 总线

二 工作原理

寄存器 数据和指令

只能根据 地址 取

因此所有东西进CPU之前一定要存在内存中

不成比例扩展效应

三 回顾 CPU 中央处理单元

{ PC — 程序计数器
ALU — 算术逻辑单元

寄存器

一些问题: 1° CPU 的频率不能无限提高 ← 改进 CPU 结构性能 指令

2° 内存墙 的存在: 主内在 CPU 之间传输数据的速度赶不上 CPU 的速度

速度差

采用高速缓存

Cache: 高速缓冲存储器 (属于CPU)
(也可增大总线数据宽度)

3° CPU等待 I/O 设备传输数据 ← 中断机制

硬盘是外部设备 属于 I/O

(设备)

中断检测: 将中断周期加入指令周期

多重中断 { 顺序中断处理
嵌套中断处理

四 回顾存储器

问题 4° 兼顾存储容量 速度 成本 ← 层次式存储结构

五 回顾 I/O (模块)

与 CPU 和内存交换外部来源收集的数据

区分 I/O 模块 I/O 设备

不属于计算机

问题 5° I/O 设备传输速率差异大 { 慢

快 接收不了

↑

采用缓冲区 改进 I/O 操作技术 新的接口技术

六 回顾总线

共享传输介质

特点 共享 ← 简化互连布局和外处理器控制

问题 6° 计算机部件互连复杂

解决

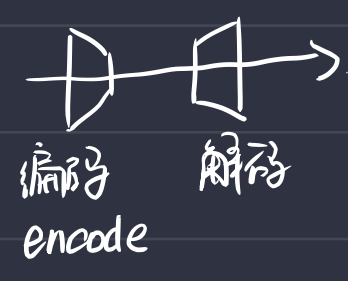
- 控制线
- 地址线
- 数据线

第三讲 数据的机器级表示

一 信息的二进制编码

冯·诺依曼中所有的信息 (代码和数据)

用基本符号对信息进行一定规律的组合
信息不丢失或少量丢失



无限连续的数据 → 离散有限

二. 整数的二进制数表示

无符号整数

有符号整数

补码

原码

反码

移码

普遍使用的

好处: 无论同号还是异号都能直接相加

补码中 $2 + (-2) = \underbrace{1000 \dots 0}_{2^n}$

0	0	-	01
1	0	-	10
↓	0	-	11
	⋮		
$2^{k-1}-1$	0	1111	11
-2^{k-1}	1	0000	00
	⋮		
-1	1	1111	11

补码的真值 $X = -X_n \times 2^{n-1} + X_{n-1} \times 2^{n-2} + \dots + X_1 \times 2^0$

范围 $-2^{k-1} \sim 2^{k-1}-1$

PPPT上形象的图片 小旗子

三. 浮点数的二进制数表示

1. 定点表示法. 表示值的范围有限

2. 方法: 科学计数法, $\pm S \times B^E$

3. 规格化数 (同一个数的表示方法唯一, 否则同一个数占用多个字符串)

$1 bbb \dots b \times 2^E$

5) 第一位是1, 于是忽略

移码: E是真实值加127

B默认是2

$E: 0 \sim 255$

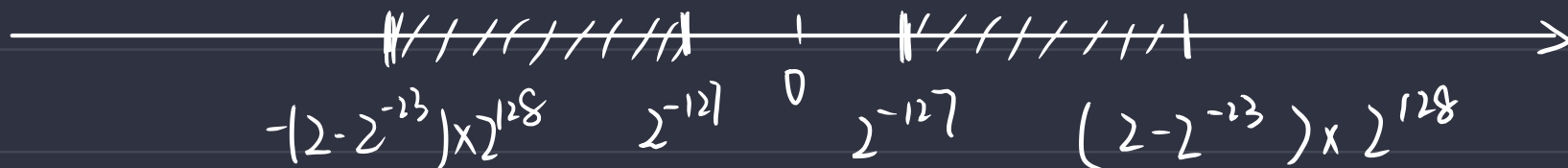
$e: -127 \sim 128$

4. $\overbrace{1.000 \dots 0}^{23 \text{ 位}} \sim \underbrace{1.111 \dots 1}_{2^{-23}}$

负上溢

环溢 正下溢

正上溢



5. 一定长度的规格化数

① 阶码 E 位数 ↑

表示范围扩大. 表示精度降低

② 尾数 S 位数 ↑

表示精度提高. 表示范围减小

③ 底 B ↑

表示范围变大. 降低表示精度

旗子的间隔变大了

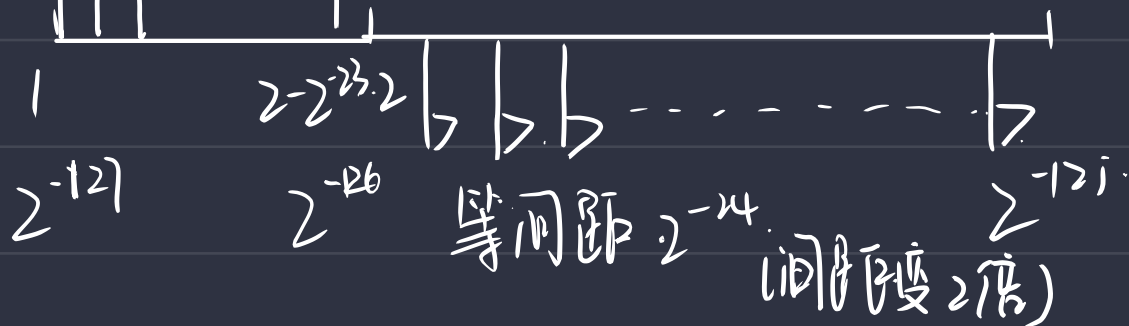
(原理) -23

等间距 2^{23} 般

PPP ... P

$$\frac{2^{127}}{2^{23}} = 2^{104}$$

IEEE 标准



$$2^{-127} \sim 2^{23}$$

float 比 int 密

$$2^{23} \sim 2^{24}$$

float 与 int 恰好一一对应

$$2^{24} \sim 2^{128}$$

float 比 int 疏

问题 float 怎么表示 0?



$-2^{-126} \sim 2^{127}$ 等间距

$-2^{-126} \sim 2^{-126}$ 非规格化数
 $2^{-126} \sim 2^{-127}$ 规格化

$$1. b b b b \dots b \times 2^{-127}$$

↓

$$0. b b b \dots b$$

→ 还需在是日里

E 不能全 1

$$2^{-23} \times 2^{-126} \sim 2^{127} (2 - 2^{-23})$$

$$= 2^{128} - 2^{104}$$

6. IEEE 754 标准

32位 单精度

1 8 23

64位 双精度

1 11 52

S 全 0 全 1

$+\infty / -\infty$

S 不全 0 全 1

NaN

E 全 0

$$0.f \times 2^{-126}$$

(牺牲了 2^{23} 个数?)

四. 二进制编码的十进制数表示

BCD码 / 8421码

0~9: 0000 ~ 1001

$\left\{ \begin{array}{l} 1100 \text{ 正} \\ 1101 \text{ 负} \end{array} \right.$

第四讲 数据校验码

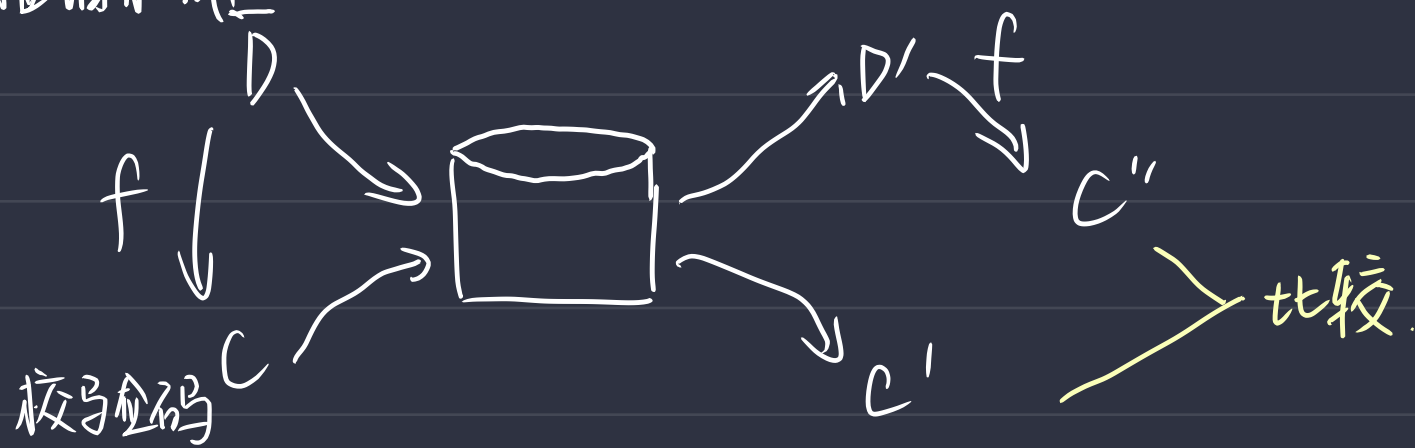
一. 差错 Error

硬故障 永久性的物理故障 某存储单元不能可靠地存储数据

软故障 电源问题 或粒子引起

二. 纠正 Error Correction

检错和纠正



基本思想：存储额外的信息以进行检错和校正

1. 奇偶校验码

1位校验码：表示1的个数是奇是偶

	奇	偶
奇校验	0	1
偶校验	1	0

$$0 \oplus 0 = 0$$

$$1 \oplus 0 = 1$$

$$1 \oplus 1 = 0$$

$$0 \oplus 1 = 1$$

$$C = D_m \oplus \dots \oplus D_2 \oplus D_1 \oplus 1$$

$$C = D_m \oplus \dots \oplus D_2 \oplus D_1$$

奇校验多异或一个“1”

数据+校验码中1的个数

同C'

奇校验 1有奇数个
偶 1有偶数个

$$S = C' \oplus C'' = 0 \Rightarrow C' \text{与} C'' \text{相等}$$

D'与D相等

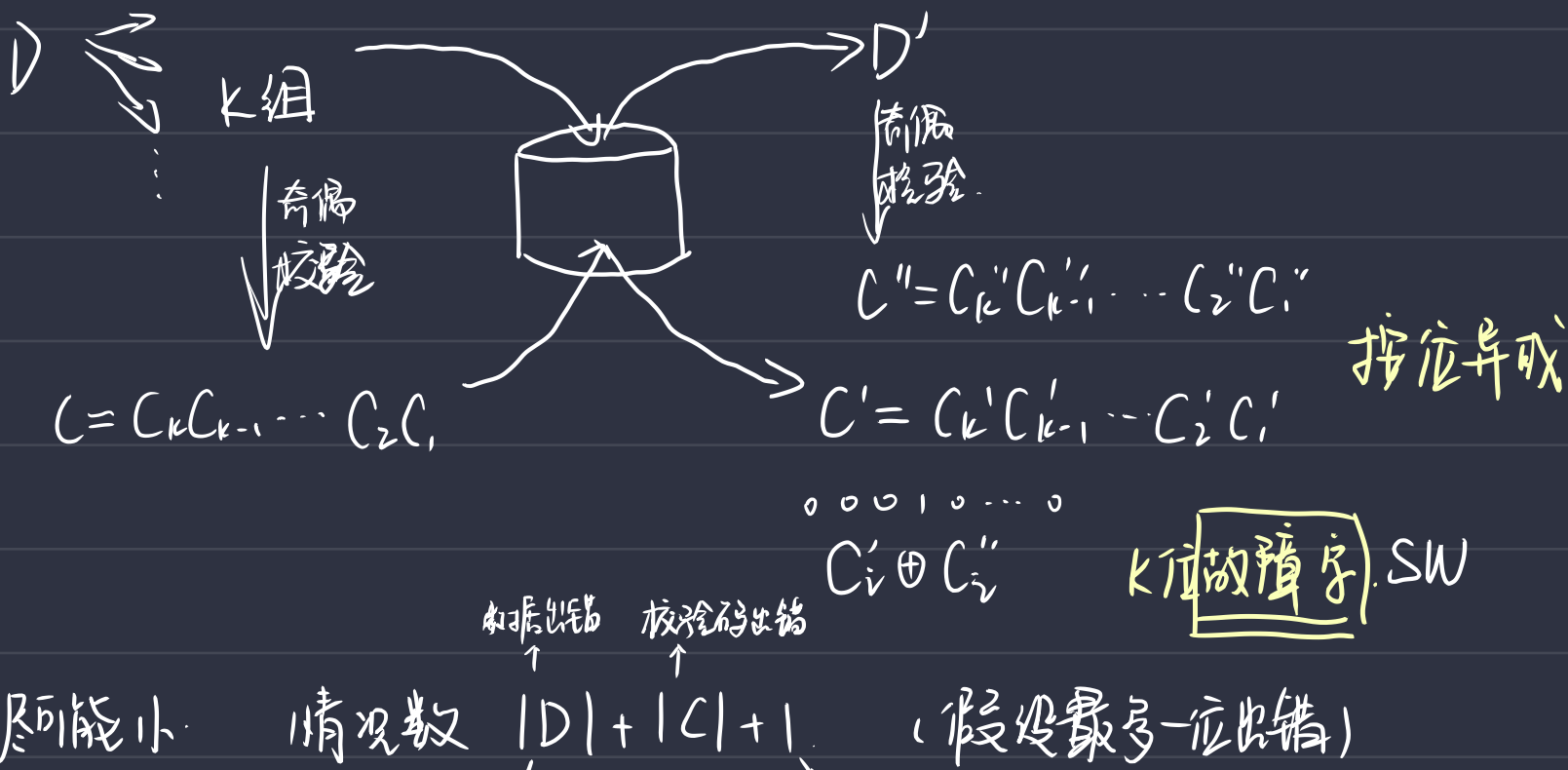
S=0 出错偶数位

S=1 出错奇数位

- 优点
 - 代价低（只需要1位额外数据，计算简单）
- 缺点
 - 不能发现出错位数为偶数的情形
 - 发现错误后不能校正
- 适用于对较短长度（如1字节）的数据进行检错

2. 海明码

基本思想 将数据分成K组 对每一组使用奇偶校验码进行检错



故障字的长度 $M + k + 1$ 没出错

$$2^k \geq M + k + 1$$

如何分组 SW 全0 000 ... 00 1个 $\Rightarrow$ 没有出错

1个 100 ... 00
010 ... 00
000 ... 10
000 ... 01 } k个 $\Rightarrow C'$ 出错 $C'_k, C'_{k-1}, \dots, C'_1$

多个1 以 8位数据 ($k=4$) 为例

D'_1 出错 0011 D_1 在 I II 组中

D'_2 0101 I III

D'_3 0110

D'_4 0111

D'_5 1001

D'_6 1010

D'_7 1011

D'_8 1100

数据位划分

- 假定数据位为8位 $D = D_8 \dots D_2 D_1$, 校验码为4位 $C = C_4 C_3 C_2 C_1$
- 数据位/校验码与故障字的关系

故障字	1100	1011	1010	1001	1000	0111	0110	0101	0100	0011	0010	0001
数据位	D8	D7	D6	D5		D4	D3	D2		D1		
校验位					C4				C3		C2	C1

数据位划分

$$C_1 = D_1 \oplus D_2 \oplus D_4 \oplus D_5 \oplus D_7$$

$$C_2 = D_1 \oplus D_3 \oplus D_4 \oplus D_6 \oplus D_7$$

$$C_3 = D_2 \oplus D_3 \oplus D_4 \oplus D_8$$

$$C_4 = D_5 \oplus D_6 \oplus D_7 \oplus D_8$$

① 通过 D 生成 C D 和 C 一起传
变成 D' 和 C'

② 通过 D' 生成 C'

③ $S = C' \oplus C$

通过 S 判断出错位置

组

I $D_1 D_2$ $D_4 D_5$ D_7

II D_1 $D_3 D_4$ $D_6 D_7$

III $D_2 D_3 D_4$ D_8

IV $D_5 D_6 D_7 D_8$

$$C_1 = D_1 \oplus D_2 \oplus D_4 \oplus D_5 \oplus D_7$$

扩展 SEC-DED (没讲)

高位: 数据

3 循环冗余校验

流格式存储

大量数据

生成多项式?



$K + D'$ 可被生成多项式整除

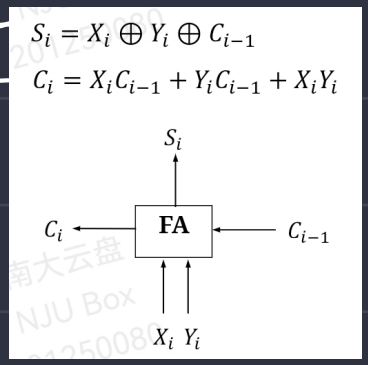
第五讲 整数运算

一. ALU 算术逻辑单元

1. 加法



全加器



与 或门延迟: 1级门延迟 (ty)
异或

6 ty
2 ty
↓
整数运算
6 ty

Q: 思考 2 3 3 2 位需时间

	$t_s (ty)$	$t_c (ty)$	$t_{total} (ty)$
$n=1$	6	2	6
2	6	4	6
3	7	6	7
4	9	8	9
$(k \geq 3) \quad n=k$	$2k+4$	$2k$	$2k+4$
$k \geq 3 \quad n=k$	$2k+1$	$2k$	$2k+1$

$n \geq 3$ 后 $t_{s_i} = t_{C_{i-1}} + 3$

$$\begin{cases} n=1,2 & t_s = 6 \\ n \geq 3 & t_s = 2n+1 \end{cases}$$

$$\Downarrow$$

$$t_s = \max(3, 2(n-1) + 3)$$

$$= \max(6, 2n+1)$$

① 串行进行加法器

② 缺点: 和 n 线性, 慢

(全) 先行进位加法器

超前进位: $P_i = X_i + Y_i \quad G_i = X_i Y_i$

$$\begin{matrix} 1 & + & 2 & + & 3 & = & 6 \text{ ty} \\ \uparrow & & \uparrow & & \uparrow & & \\ P_i & & Q_i & \Rightarrow & C_i & \Rightarrow & S_i \end{matrix}$$

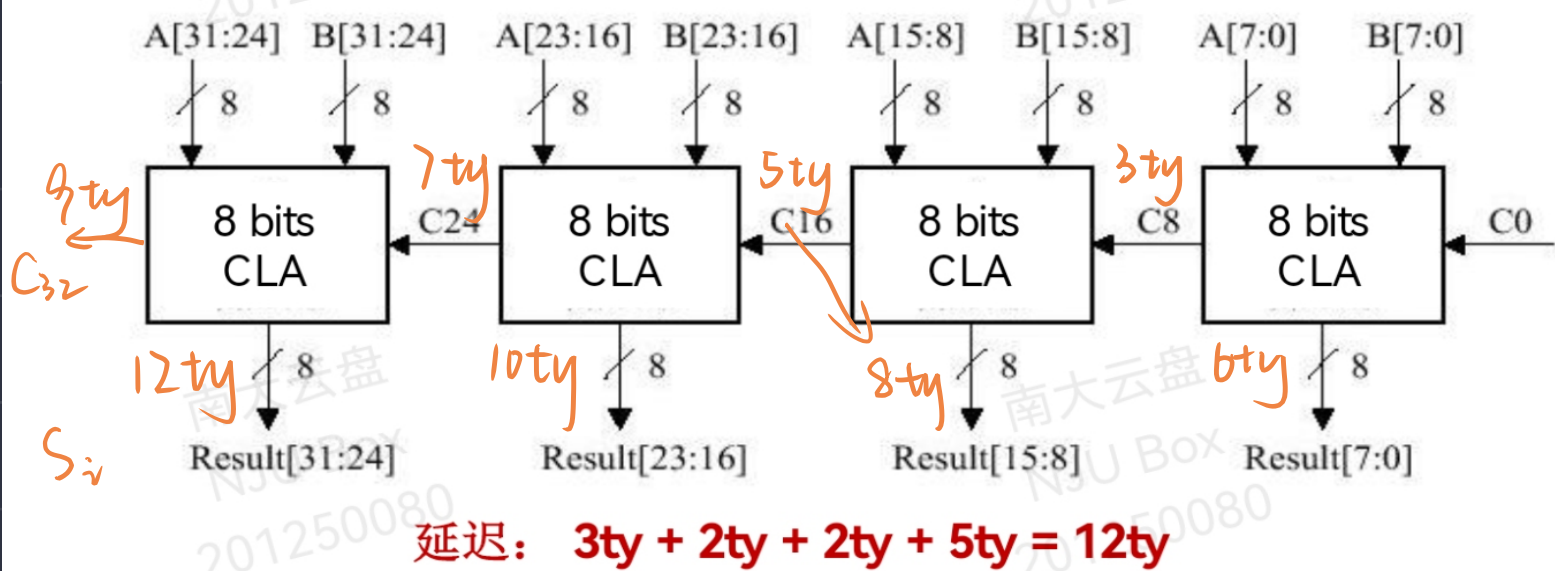
与 n 无关
缺点: 复杂

③ 部分先行进位加法器

思路

- 采用多个CLA并将其串联，取得计算时间和硬件复杂度之间的权衡

例子



$$\begin{cases} t_{C_{8k}} = 2k + 1 \\ t_{S_{8k}} = 2k + 4 \end{cases}$$

k组先行进位加法器

加法器溢出

判① $X_n = Y_n$ 且 $S_n \neq X_n$ 符号位

断 $overflow = X_n Y_n \bar{S}_n + \bar{X}_n \bar{Y}_n S_n$

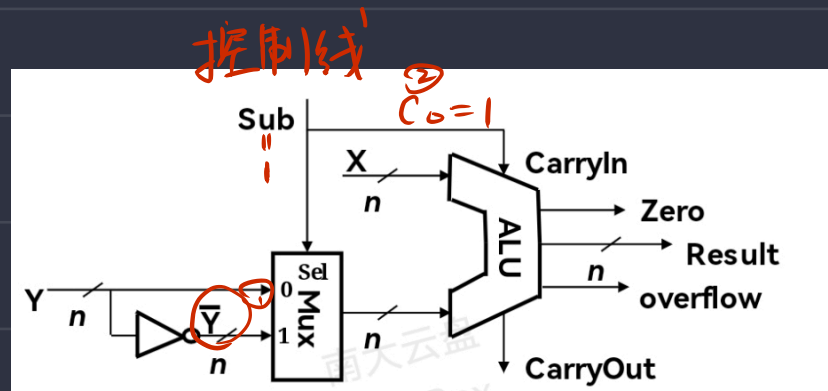
② $C_n \neq C_{n-1}$ 最后两个进位

$$overflow = C_n \oplus C_{n-1}$$

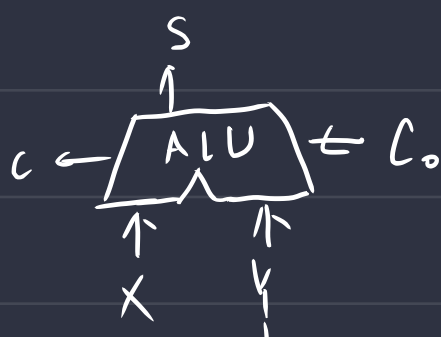
2. 减法

$$X - Y = X + (-Y)$$

取反 加一
↑ ↑
取非 设 $C_0 = 1$



3. 乘法

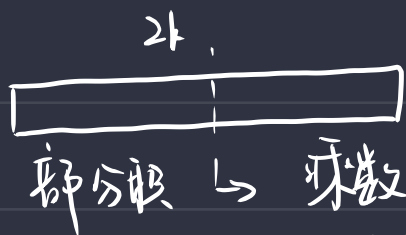


① k位初始化为全0

② (加) 右移补0

↓
全0 跳过
倍数数

实现:



负数乘法出错 $\Rightarrow$ 布斯算法

1.0, -1.1

①增加%

①根据 $Y_t - Y_{t-1}$ 决定 $+X / +(-X) / 0$ 以上

②右移,补已有符号位

思考: 进位中产生的进位为什么扔掉?

4. 除法 (不完全正确) 除数与被除数都不为0的情况

被除数 $\rightarrow$ 余数

除韵

商

① 347432 000 0kkkk

KKKK0000 0000

② 除数右移. 判断余数是否够减除数.

日多, 减则减 商左移补1

↑ 次

② 左上位为余数

2n位 ALU

↓

n 位 ALU

实现

除数左移 $\rightarrow$ 余数左移



余数 $\leftarrow 1$



余数

商

除号位
破除数 $\rightarrow$ 余数

商

陈数

① 347432 0000kkkk

k k k k

② 在数左添 判断合数是否可多减除数

{ 同增减则减 商左移补1

4 次

除数始终未变

余数 ~~减~~除数 不能变号

变号则恢复

中间系数 $\begin{cases} \text{同号则减} \\ \text{异号则加} \end{cases}$

“恢复余数”

细节: 除数和被除数异号时, 商要取反加一

“不恢复余数”

$$R_i = 2R_{i-1} - Y$$

$$R_{it+1} = 2R_{it} - Y$$

$$R_{i+1} = 2R_i + 1$$

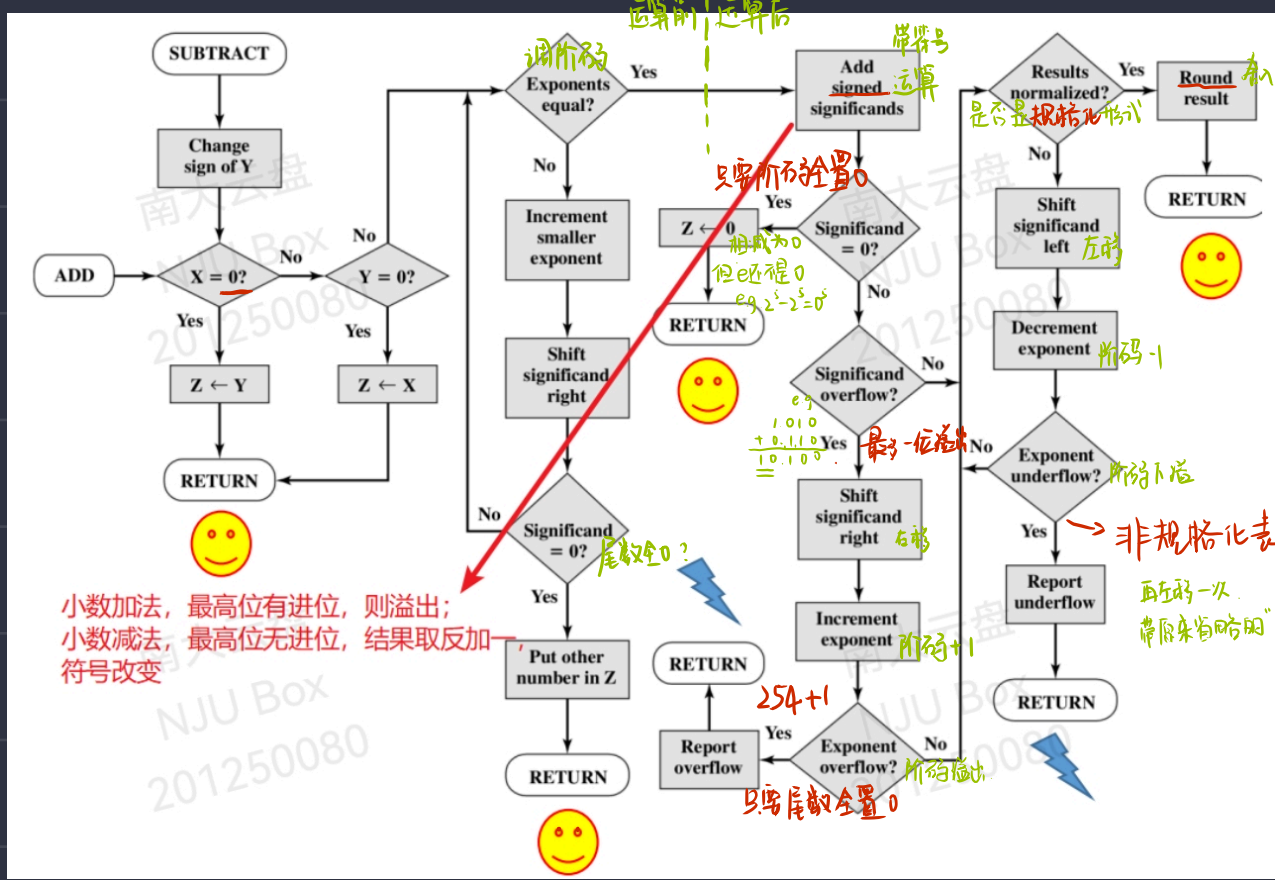
回来看 PP1 例子 和 详细规则

一 加法和减法

阶码一致 才能对尾数相加

又忽略的“1”哦：参与运算

对阶: 小的向大的靠拢


$$1.0101 \times 2^{-127}$$

$$0.10101 \times 2^{-126}$$

e.g. $\begin{matrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & \dots & 0 & X \\ \Rightarrow & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & \dots & 0 & \checkmark \end{matrix}$

求补

$$-1 \dots \times 2^e$$

$$= 2 \times 2^e - 0 \dots \times 2^e$$

$$01825 - 0.625 \rightarrow \text{取反} + 1$$

$$= 01825 + 0.375 = 1$$

??

还不进就取反+1

P10

1.1100...0

→ 右移

0 11100...0

0.00011...1

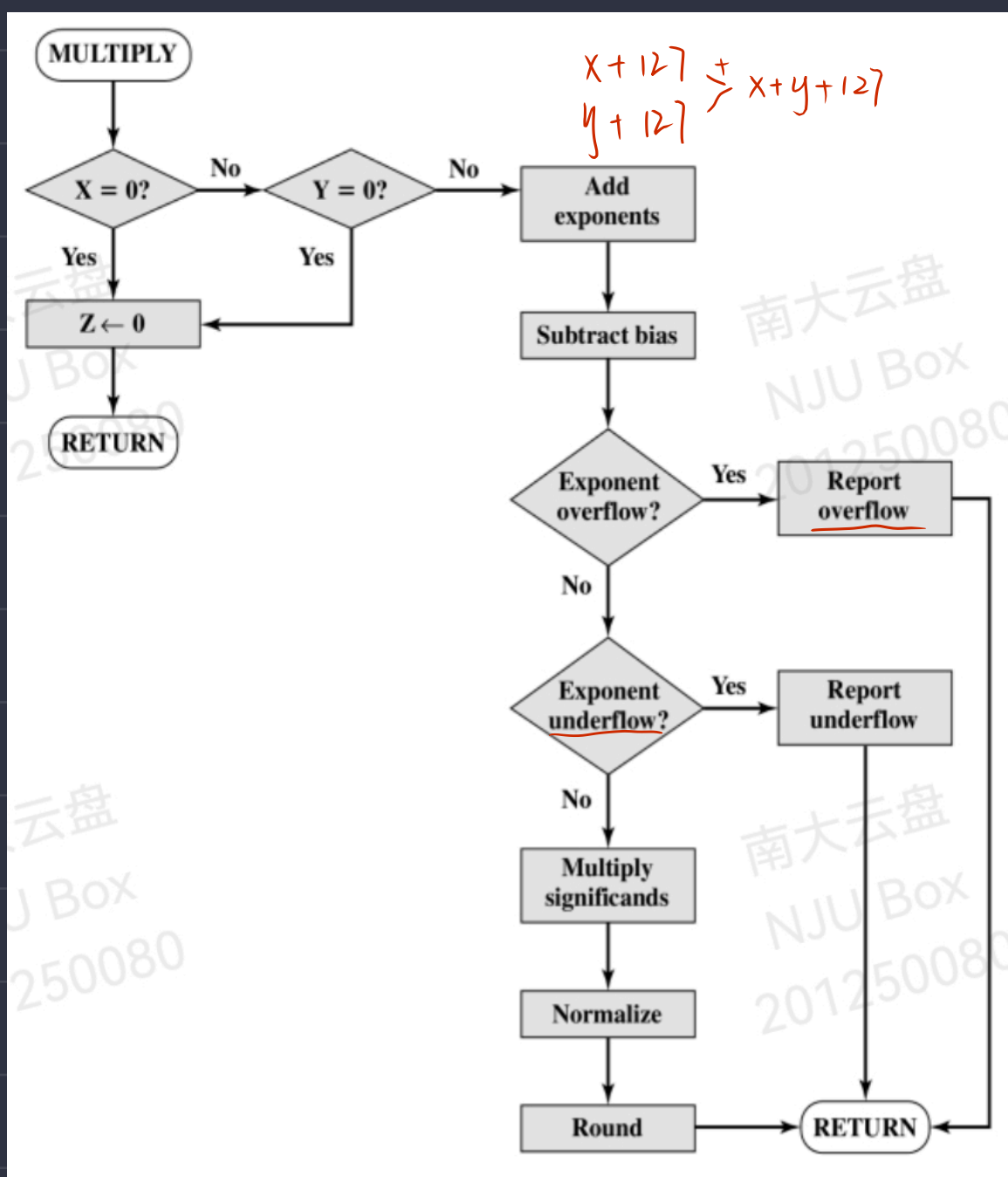
0.00100...0

② 取反+1

11000000 2x2^e

10

二. 乘与除



三. 精度考虑

前2次作业对Ans

1. ALU 32位运算

保护位

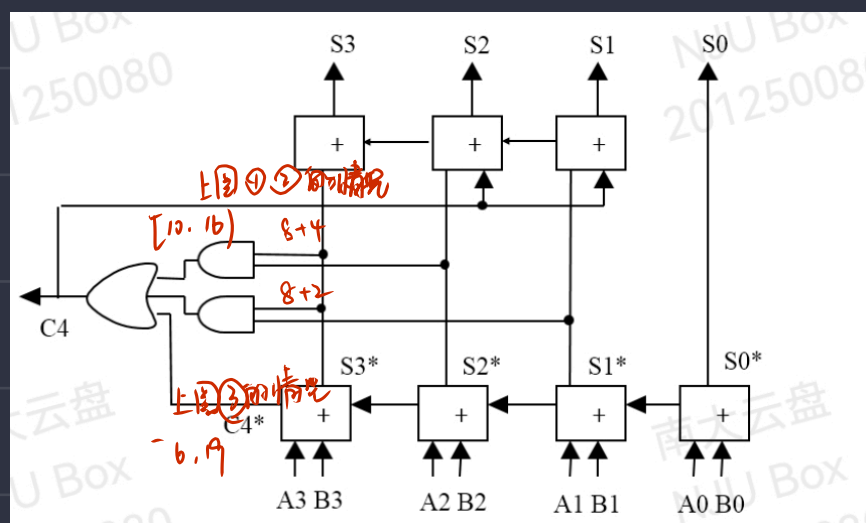
2. 舍入 朝+∞舍入 正进位 负舍位

朝0舍入 舍位

就近舍入 第一个保护位若1则进; 若0则舍

第七讲 二进制编码的十进制数的运算 NBCD

加法		
$25 + 31 = 56$ ① 0010 0101 + 0011 0001 ----- 0101 0110 $25 + 39 = 64$ ② 0010 0101 + 0011 1001 ----- 0101 1110 + 0110 ----- 0110 0100	$28 + 39 = 67$ ③ 0010 1000 + 0011 1001 ----- 0101 0001 + 1 ----- 0110 0111	$98 + 79 = 177$ ④ 1001 1000 + 0111 1001 ----- 0000 0001 + 1 1 ----- 0110 0110 0001 0111 0111



加法

① 按位相加时

若有进位, 则向上一位加1

并在A位加"0110"

进位

② 若无进位, 但也超过31001 (9)

$$9 - x = (15 - x) - 6 = 15 - (x + 6)$$

取反

复习时没有懂
但应该不会考硬件实现吧...

2. 减法 $x - y = x + (-y)$

取反 + 10 (1010)

+8 取反 (0110)

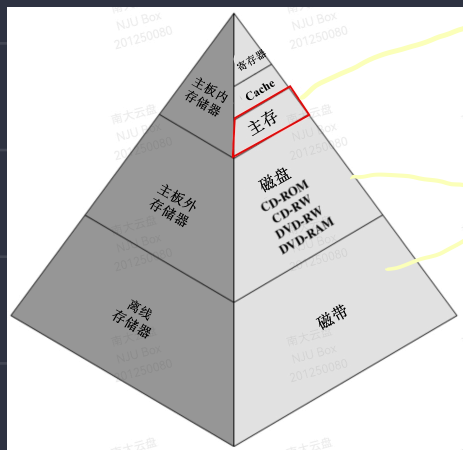
第8讲 内部存储器

(一) 层次结构

地址空间：可唯一标识的单元总数

寻址能力：存储在每个单元中的信息的位数

- 大多数存储器是字节寻址的，而执行科学计算的计算机通常是64位寻址的 **8 bit**



冯·诺依曼结构中的存储器
Memory / Inner Memory

不属于冯·诺依曼结构
外部存储器 是I/O设备

(二) 半导体存储器

位元 存储1位

特性 { 2种稳态
至少被写入1次
可被读取

一. 随机存取存储器 RAM

易失性 Volatile 断电即无

{ 动态 DRAM 电容器 周期性刷新
静态 SRAM 触发器 逻辑门

- DRAM比SRAM具有更简单、更小的位元，但要求能支持刷新的电路
- DRAM比相应的SRAM密度更高，价格更低
- SRAM通常比DRAM快
- DRAM更倾向于满足大容量存储器的需求，SRAM一般用于高速缓存，DRAM用于主存

cache

耗时慢 但都易失

存储器类型	种类	可擦除性	写机制	易失性
随机存取存储器 (RAM)	读-写存储器	电可擦除，字节级	电	易失
只读存储器 (ROM)	只读存储器	不可能	掩膜	非易失
可编程ROM (PROM)			电	
可擦除PROM (EPROM)	紫外线可擦除，芯片级			
电可擦除PROM (EEPROM)	电可擦除，字节级			
快闪存储器	电可擦除，块级			

高级 DRAM 结构

1° SDRAM 同步 DRAM

异步 → 同步 时钟周期上升沿

2° DDR SDRAM 双数据速率

上升沿和下降沿 每时钟周期发送2次数据

二. ROM 只读存储器

非易失 可读 不能写入新数据 无出错机会 在出厂时设置好 掩膜

→ PROM - 可编程 ROM

ROM: 大批量生产领域仍有吸引力

写: 电信号执行

三. Read-Mostly Memory 主要进行读操作的存储器
非易失. 写、读较困难

→ 1° EPROM erasable programmable

· 光擦除 紫外线 20min 全部擦除

· 电写入

2° EEPROM electrically

· 电可擦除

更新以字节为单位

集成度低 支持小容量芯片

3° Flash Memory 闪存存储器

SSD 硬盘. U 盘

· 电可擦除

· 速度 EPROM 与 EEPROM 之间

· 块级擦除

· 集成度 同 EPROM

二. 内存相关

内存条?

0.

1. 寻址单元 由若干相同地址的位元构成

寻址模式 字节 Byte 常用

字 Word

2. 存储阵列 由大量寻址单元 构成

2048 × 2048 × 4

行 列 每个存储4 bit

行译码器 2048根线

列译码器 2048根线

地址译码器在计算机中，
好熟悉。

刷新：

① 集中式刷新

② 分散式刷新

③ 异步刷新

相当于增加3个存储周期的时间（考虑内存墙）

常用

刷新时间 刷新需要的时间

每2行刷新时间间隔时间

刷新周期 对于每一行 两次刷新时间
间隔时间

3. 芯片

封装存储阵列和引脚

4. 模块组织

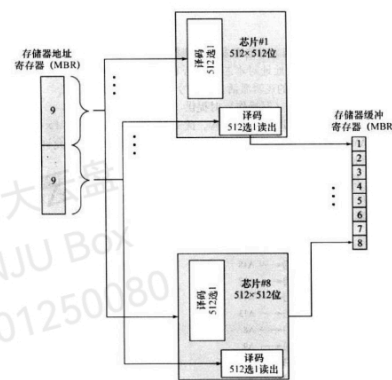
1) 位扩展

2) 字扩展

3) 位字同时扩展

模块组织

- 位扩展：地址线不变，数据线增加
• 使用8块 4K*1 bit 的芯片组成 4K*8 bit 的存储器 寻址单元变大
- 字扩展：地址线增加，数据线不变
• 使用4个 16K*8 bit 的芯片组成 64K*8 bit 的存储器 寻址单元个数变多
- 字、位同时扩展：地址线增加，数据线增加
• 使用8个 16K*4 bit 的芯片组成 64K*8 bit 的存储器



5. 插槽

- 一般是字扩展

△6. 地址是起始地址

只有一种



12
36
56
78

78
56
36
12

大端序

小端序

不是87

字节反

而不是 二进制序反

第9讲 高速缓冲寄存器 Cache

一. Cache 设计思路

1. 问题 内存墙 Memory Wall

2. Cache 基本思路 “预判”

主存信息的 copy

但可能存在不同的情况: CPU 写到 Cache, 但还没写到主存

3. Cache 工作流程

检查 check { 命中 hit
未命中 miss

是固定的划分

将主存中已含该字的固定大小的块 block 读入 Cache 中

再从 Cache 传字给 CPU

对几个问题的解释

① 判断 Hit / Miss ?

tag

→ 标识在主存中的对应位置

② 通过 Cache ? block ?

程序访问的局部性原理

时间局部性

空间局部性 - 顺序局部性等

Cache 存取都是以 block 为单位

③ Cache 操作更多, 仍节省时间

命中率 P

访问 Cache 的时间

$$T_A = P \times T_C + (1-P)(T_C + T_m) = T_C + (1-P)T_m$$

$$T_A < T_m \Rightarrow P > T_C / T_m$$

二. Cache 的设计要素

1. cache 容量

主存块到 Cache 行的对应

2. 映射功能 Mapping Function

主存块 $\rightarrow$ cache 行

该行影响 cache 组织结构

① 直接映射

固定对应关系, 取模操作 (模行数)

标识	Cache 行号	块内地址
----	----------	------

存在某一行块地址结尾相同 标识位与块号短

Direct

$$T_C^D = t_0$$

缺: 抖动现象

优: 快速映射, 快速检查 (T_C 是固定的) 不随行数增加变长

适用: 大量量的 cache

② 关联映射 Associate 命中
 $T_c^A = (1-p) C t_0 + p \frac{1+C}{2} t_0$
check 时间

适用: 小容量 cache

③ 组关联映射

固定组 任意行

↓
对组数取模
S

k 每组的行数

k-路组关联映射

S = C 标记的长度: S = 1
k = 1 k = C

直接映射 < 组关联 ... < 关联 ...

块号 - |mod C| 块号 - |mod S| 块号长度
set

$$T_c^S = (1-p) k t_0 + p \frac{1+k}{2} t_0$$

△ 关联度: 数值上等于 k

块可以访问的行数

关 ↓

命中率 ↓

check 的时间 ↓

标记所占的额外空间开销 ↓

块号与 tag

P

$$|tag| = |块号| - \lceil \log_2 C \rceil$$

A

$$|tag| = |块号|$$

S

$$|tag| = |块号| - \lceil \log_2 S \rceil$$

x mod C

新块与将被替换的组内

3 替换算法 (直接映射无需考虑) 前 2 个常考

1° 最近最少使用 LRU 某行 对应关系

依据: 最长时间未被使用 (访问)

实现: 以 2 路组为例 包含 USE 位

△ 每一次替换

都是由未命中引起的

2° 先进先出算法 FIFO

依据: 在 Cache 中停留时间最长

实现: 时间片轮转法 or 环形缓冲技术

放入行顺序固定

3° 最不经常使用算法 LFU

4° 随机替换算法 Random

4. 写策略 (从 Cache 写入主存. "同步")

1° 写回法

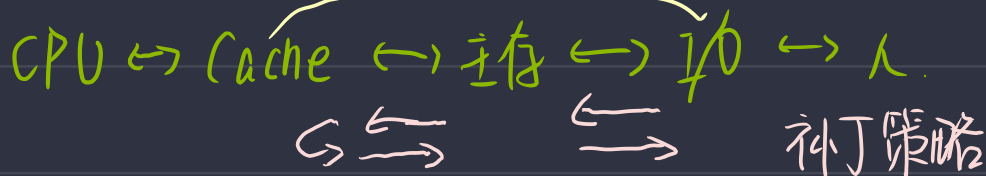
被替换时 判断是否修改 写主存 (不一定)

脏位 (dirty bit)

使用位 (use bit)

缺: 主存中数据可能不是最新的

优: 减少对主存的访问



2° 写直达

减慢写操作 不影响读操作

优: 同步, 数据最新

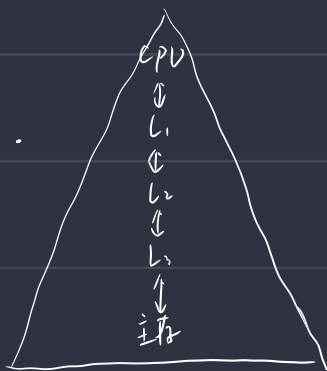
5. 行大小

和命中率关系复杂

总体趋势: $\cap$

6. Cache 数目

① 多级



计算 TC:

① 不同级命中 相加

② 按流程

② 统一 vs. 分立

第十讲 外部存储器 (磁盘 磁带)

不是 I/O 模块 不属于冯诺依曼结构
是 I/O 设备

和内部存储器相比的最大特点是：
非易失性

一、磁盘存储器

1 结构 { 可磁化材料 非磁化材料 (基材) (一般是玻璃 玻璃不易变形)

磁头 { 带电磁场
飞行高度

→ 磁头悬空

① 写磁化 ② 读感应电流 ③ 屏蔽

磁道

双磁头

→ 读取磁头

MR 磁阻感应器

磁场 → R → I → U

2 数据组织

磁道 track

扇区 sector

传入传出的单位 默认 512B

题目可能不给

划分 { 恒定角速度

CAV Constant angular velocity

优 以磁道和扇区号定位

缺 最内层磁道最大记录密度 限制 磁盘存储容量

多带式 / 多重区域记录

Multiplex zone recording

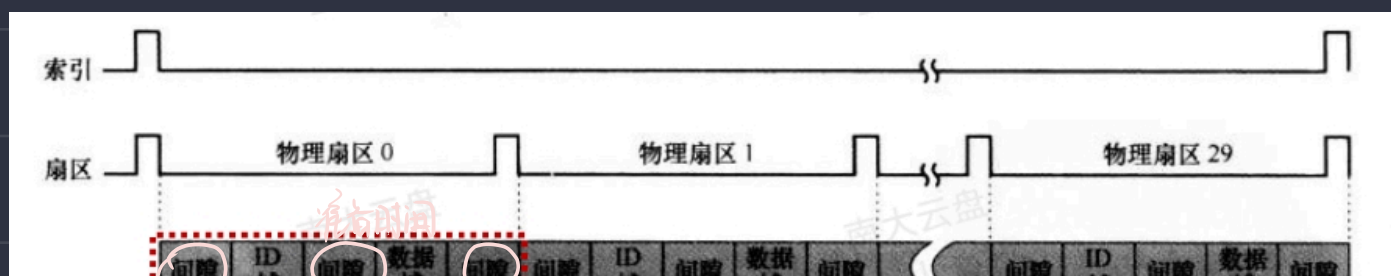
(阶梯状增加)

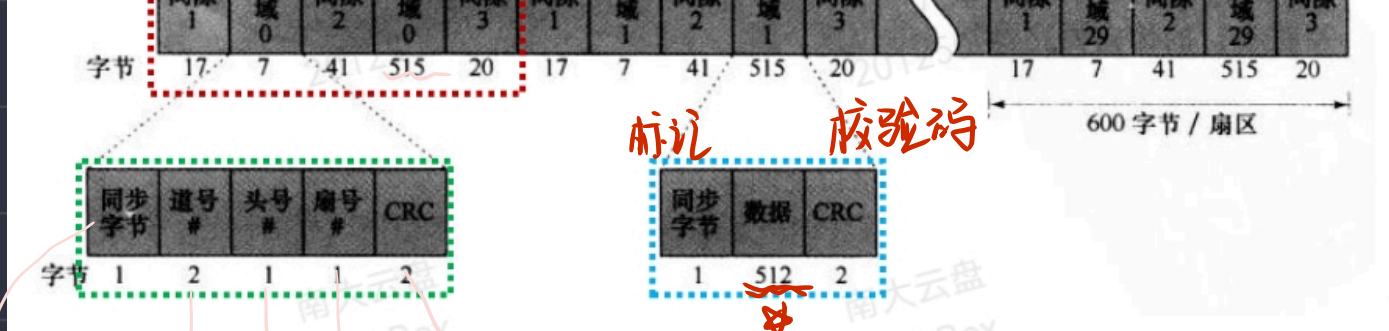


变速原因 磁盘读写速度与半径成正比

间隙 gap

⇒ 柱面 cylinder 相同位置的磁道组





标记
准备时间
柱面 磁头 扇区 校验
(磁道)

3. 格式化

4. I/O 访问时间

寻道时间：磁头定位磁道

旋转延时：扇区定位磁头

传送时间 $T = \frac{b}{rN}$

T = 传送时间
 b = 传送的字节数
 N = 每磁道的字节数
 r = 旋转速率, 单位是转/秒

rpm 每分

$$T_a = T_s + \frac{1}{2r} + \frac{b}{rN}$$

连续访问多个磁道时：一次寻道时间 + 每次旋转延时

硬磁盘存储器：I/O 访问时间 (示例)

- 假设某个硬盘的平均寻道时间为4ms, 转速为15000rpm, 每磁道500扇区, 每扇区512B, 现读取一个由2500个扇区组成的文件
- 情况 1: 顺序组织
 - 该文件占据相邻5个磁道的全部扇区 (5 磁道 $\times$ 500 扇区/道 = 2500 扇区)
- 情况 2: 随机存取
 - 该文件随机分布在磁盘上的各个扇区

情况 1: 顺序组织

平均寻道	4 ms
平均旋转延迟	2 ms
读500个扇区	4 ms
	10 ms

访问后续磁道不计寻道时间, 因此访问每一个后续磁道的用时为 $2 + 4 = 6$ ms

总时间 = $10 + (4 \times 6) = 34$ ms = 0.034 s

情况 2: 随机存取

平均寻道	4 ms
平均旋转延迟	2 ms
读1个扇区	0.008 ms
	6.008 ms

总时间 = $2500 \times 6.008 = 15020$ ms = 15.02 s

↓
磁盘整理

↓
SSD

5. 磁头寻道/磁盘调度 (多个读写请求)

减少寻道时间

① 先来先服务 FCFS

② 最短寻道时间优先 SSTF

饥饿现象

③ 扫描/电梯算法 SCAN

④ 循环扫描 C-SCAN

⑤ LOOK

⑥ C-LOOK

无饥饿现象 各磁道响应频率不均匀 (波动性)
频率平均 但平均寻道时间↑

* 最外圈磁道是0号

(不是越后寻道位置)
* 关心的是读写结束时的磁头位置, 以决定下次

二 光存储器

大概分3类: CD DVD 蓝光

激光细
数据密度高

1. CD

CD-ROM 耐用 纠错功能 激光在凹坑和凸土反射

2. 单螺旋轨道 不是同心圆 所有扇区长度相同

线速度恒定

3. CD-R 只能写一次

CD-RW 可写多次

4. DVD

蓝光盘

三 磁带

1. 读取特点: 顺序读取 (磁盘: 直接读取)

{ 并行记录 (多磁头)

串行记录 (蛇形记录) $\Rightarrow$

四 快闪存储器 $\rightarrow$ U盘 非易失

固态硬盘SSD 抗干扰性好

总结

• 磁盘存储器

• 软盘

• 硬盘: 结构, 读写机制, 数据组织, 格式化, I/O访问时间, 磁头寻道

• 光存储器

• CD, CD-ROM, CD-R, CD-RW

• DVD, DVD-R, DVD-RW

• Blu-ray

• 磁带

• U盘和固态硬盘

第十一章 RAID 冗余磁盘阵列

(独立 不是完全独立)

逻辑上是单个逻辑驱动器

物理上使用一组物理磁盘驱动器

RAID分类						
种类	级别	描述	磁盘要求	数据可用性	大 I/O 数据传输能力	小 I/O 请求速率
条带化	0	非冗余	N	比单盘低	很高	读和写都很高
镜像	1	镜像	$2N$	比 RAID 2、3、4、5 高；比 RAID 6 低	读比单盘高；写与单盘类似	读高达单盘的两倍；写与单盘类似
并行存取	2	汉明码冗余	$N+m$	比单盘高很多，与 RAID 3、4、5 差不多	列表各级中最高	接近于单盘的两倍
	3	位交错奇偶校验	$N+1$	比单盘高很多；与 RAID 2、4、5 差不多	列表各级中最高	接近于单盘的两倍
独立存取	4	块交错奇偶校验	$N+1$	比单盘高很多；与 RAID 2、3、5 差不多	读与 RAID 0 类似；写低于单盘	读与 RAID 0 类似；写显著低于单盘
	5	块交错分布式奇偶校验	$N+1$	比单盘高很多；与 RAID 2、3、4 差不多	读与 RAID 0 类似；写低于单盘	读与 RAID 0 类似；写显著低于单盘
	6	块交错分布式奇偶校验	$N+2$	列表各级中最高	读与 RAID 0 类似；写比 RAID 5 低	读与 RAID 0 类似；写显著低于 RAID 5

一. RAID 0 非冗余

图中每个“桶”代表一个硬盘

- 数据以条带的形式在可用的磁盘上分布
- 不采用冗余来改善性能（不是RAID家族中的真正成员）
- 用途
 - 高数据传输率
 - 高速响应I/O请求

写/读时分散到各硬盘
同时寻道（2个I/O请求需要的数据块不同磁盘上）



缺点：1坏则全不能用

二. RAID 1 镜像

约等于“RAID 0 x 2”

- 优点
 - 高速响应I/O请求：即便是同一个磁盘上的数据块，也可以由两组硬盘分别响应
 - 读请求可以由包含请求数据的两个对应磁盘中的某一个提供服务，可以选择寻道时间较小的那个
 - 写请求需要更新两个对应的条带：可以并行完成，但受限于写入较慢的磁盘
 - 单个磁盘损坏时不会影响数据访问，恢复受损磁盘简单
- 缺点
 - 价格昂贵

允许坏1~n个
2个RAID 0中的同一个

三. RAID 2 并行存取 汉明码冗余

- 目标
 - 所有磁盘都参与每个I/O请求的执行
- 特点
 - 各个驱动器的轴是同步旋转的，因此每个磁盘上的每个磁头在任何时刻都位于同一位置
 - 采用数据条带：条带非常小，经常只有一个字节或一个字

允许坏2~3个
(b及其校验位) (校验位冗余)

四. RAID 3 并行存取 位交错奇偶校验

本身不能纠错

解决磁盘损坏问题

不用于定位 用于恢复数据

$$p(b) = b_0 \oplus b_1 \oplus b_2 \oplus b_3$$

$$b_0 \oplus p(b) = b_0 \oplus b_0 \oplus b_1 \oplus b_2 \oplus b_3 = b_1 \oplus b_2 \oplus b_3$$

$$b_0 = b_0 \oplus p(b) \oplus p(b) = p_0 \oplus b_1 \oplus b_2 \oplus b_3$$

• 优缺点

- 优点：能够获得非常高的数据传输率
 - 对于大量传送，性能改善特别明显
- 缺点：一次只能执行一个I/O请求
 - 在面向多个I/O请求时，性能将受损

允许坏1个

五. RAID 4 独立存取 位交错奇偶校验

- 采用独立存取技术
 - 每个磁盘成员的操作是独立的，各个I/O请求能够并行处理
- 采用相对较大的数据条带
- 根据各个数据盘上的数据来逐位计算奇偶校验条带，奇偶校验位存储在奇偶校验盘的对应条带上

• 性能

- 当执行较小规模的I/O写请求时，RAID 4会遭遇写损失
 - 对于每一次写操作，阵列管理软件不仅要修改用户数据，而且要修改相应的校验位
- 当涉及所有磁盘的数据条带的较大I/O写操作时，只要用新的数据位来进行简单的计算即可得到奇偶校验位
- 每一次写操作必须涉及到唯一的校验盘，校验盘会成为瓶颈

校验盘

都要改动校验位并写入磁盘

无论与在哪个磁盘

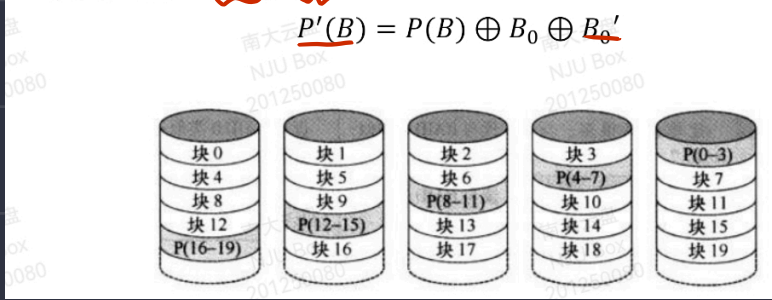
$$P'(B) = P(B) \oplus B_0 \oplus B_0'$$

六. RAID 5 独立存取 位交错分布式奇偶校验

- 与RAID 4 组织方式相似
- 在所有磁盘上都分布了奇偶校验条带
 - 避免潜在的I/O瓶颈问题
- 访问时的“两读两写”

读同时写同时
用时2~4

读或写都顺序



①读 B, P
②写 B', P'

机试

1h x1.1 91分 以离场为时限

笔试

8题

2h

7x15 min + 15 min

7 不按难度顺序 先做会的 尽量多写

1 最后一题选择 30分 3x10 建议先做. 比作业简单一点.